

Lab 4: Arithmetic Expression Evaluator Using Flex and Bison

Experiment Title

Arithmetic Expression Evaluator using Flex and Bison

Aim

To write a program using Flex and Bison that:

- Parses and evaluates arithmetic expressions.
- Supports: Addition, Subtraction, Multiplication, Division.
- Supports parentheses and integer expressions.
- Detects and handles syntax errors.

Software Requirements

Tool	Purpose
Flex	Lexical Analyzer Generator
Bison	Parser Generator
GCC	C Compiler to compile generated code
OS	Windows (MinGW)

Theory

A parser analyzes the structures of expressions using grammar rules. Bison is used to define these grammar rules, while Flex supplies tokens. This experiment demonstrates:

- Defining a context-free grammar for arithmetic expressions.
- Resolving operator precedence and associativity.
- Evaluating the result during parsing.

Program Structure

Flex File (key.l)

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}

%%

/* Match integer numbers */
```

```

[0-9]+ {
    yylval.num = atoi(yytext);
    return NUMBER;
}

/* Ignore spaces and tabs */
[ \t]+ ;

/* Return newline to parser */
\n {
    return '\n';
}

/* Return operators and parentheses directly */
[()+\-*/] {
    return yytext[0];
}

/* Handle invalid characters */
. {
    printf("Unknown character: %s\n", yytext);
}

%%

int yywrap(void)
{
    return 1;
}

```

Bison File (y.y)

```

%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);
%}

/* Define semantic value type */
%union {
    int num;
}

```

```
/* Token declarations */  
%token <num> NUMBER
```

```
/* Non-terminal value types */  
%type <num> expr term factor
```

```
%%
```

```
/* Starting symbol */
```

```
input:
```

```
    /* empty */  
    | input line  
    ;
```

```
line:
```

```
    '\n'  
    | expr '\n'  
    {  
        printf("Result = %d\n", $1);  
    }  
    ;
```

```
/* Addition and subtraction */
```

```
expr:
```

```
    expr '+' term  
    {  
        $$ = $1 + $3;  
    }  
    | expr '-' term  
    {  
        $$ = $1 - $3;  
    }  
    | term  
    {
```

```
    $$ = $1;  
  }  
  ;
```

/* Multiplication and division */

term:

```
  term '*' factor  
  {  
    $$ = $1 * $3;  
  }  
| term '/' factor  
  {  
    if ($3 == 0)  
    {  
      yyerror("Division by zero");  
      $$ = 0;  
    }  
    else  
    {  
      $$ = $1 / $3;  
    }  
  }  
| factor  
  {  
    $$ = $1;  
  }  
;
```

/* Parentheses and numbers */

factor:

```
  '(' expr ')'  
  {  
    $$ = $2;  
  }  
| NUMBER
```

```

    {
        $$ = $1;
    }
;

%%

/* Error handling function */
void yyerror(const char *s)
{
    fprintf(stderr, "Error: %s\n", s);
}

/* Main function */
int main(void)
{
    printf("=====\n");
    printf(" Arithmetic Expression Parser\n");
    printf("=====\n");
    printf("Examples:\n");
    printf(" 3+4*2\n");
    printf(" (3+4)*2\n");
    printf(" 10/5+7\n");
    printf("\nPress Ctrl+Z (Windows) or Ctrl+D (Linux) to exit.\n\n");

    yyparse();

    return 0;
}

```

Compilation Steps

```

bison -d y.y
flex y.l
gcc -o result y.tab.c lex.yy.c

```

Sample Input / Output

Input	Output
$2+3*4$	Result = 14
$(1+2) * (3+4)$	Result = 21
$10/0$	Error: Division by zero = 0

Result

- The parser correctly evaluates arithmetic expressions.
- Operator precedence (* and / over + and -) and parentheses are respected.
- Division-by-zero is safely handled.
- Syntax errors are reported clearly.

Conclusion

- Learned how to use Bison to define arithmetic expression grammars.
- Understood tokenization via Flex and how it interacts with a Bison parser.
- Built a working calculator with error handling for invalid or dangerous input.